CHAPTER 1

LOAD BALANCERS

1 Introduction to Load Balancers . . . 2
2 Global and Local Load Balancing . . 4
3 Advanced Details of Load Balancers 9

Take a closer look at load balancers, their placement, and algorithms, including local and global load balancers and different tiers of load balancers.

1 Introduction to Load Balancers

1.1 What is load balancing?

Millions of requests could arrive per second in a typical data center. To serve these requests, thousands (or a hundred thousand) servers work together to share the load of incoming requests.

> **Note:** Here, it's important that we consider how the incoming requests will be divided among all the available servers.

A **load balancer** (**LB**) is the answer to the question. The job of the load balancer is to fairly divide all clients' requests among the pool of available servers. Load balancers perform this job to avoid overloading or crashing servers.

Why do we need load balancers?

The load balancing layer is the first point of contact within a data center after the firewall. A load balancer may not be required if a service entertains a few hundred or even a few thousand requests per second. However, for increasing client requests, load balancers provide the following capabilities:

- **Scalability**: By adding servers, the capacity of the application/service can be increased seamlessly. Load balancers make such upscaling or downscaling transparent to the end users.
- **Availability**: Even if some servers go down or suffer a fault, the system still remains available. One of the jobs of the load balancers is to hide faults and failures of servers.
- **Performance**: Load balancers can forward requests to servers with a lesser load so the user can get a quicker response time. This not only improves performance but also improves resource utilization.

Here's an abstract depiction of how load balancers work:

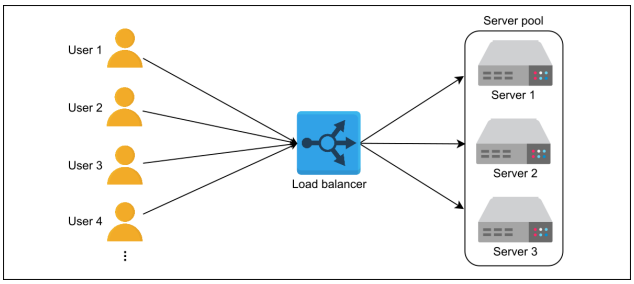


Figure 1.1: Simplified working of a load balancer

The requests received by a load balancer are distributed among multiple servers using a configured algorithm that we'll discuss in the upcoming lessons:

- Round-robin
- Weighted round-robin
- Least response time
- Least connections

Load balancers ensure reliability and availability by continuously monitoring server health and directing traffic only to servers capable of responding efficiently. We 'll explore these algorithms and their impact on system performance in more detail in the Advanced Details of Load Balancers lesson.

1.2 Placing load balancers

Generally, LBs sit between clients and servers. Requests go through to servers and back to clients via the load-balancing layer. However, that isn't the only point where load balancers are used.

Let's consider the three well-known groups of servers. That is the web, the application, and the database servers. To divide the traffic load among the available servers, load balancers can be used between the server instances of these three services in the following way:

- Place LBs between end users of the application and web servers/application gateway.
- Place LBs between the web servers and application servers that run the business/application logic.
- Place LBs between the application servers and database servers.

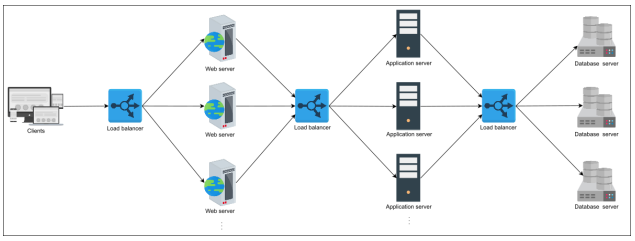


Figure 1.2: Possible usage of load balancers in a three-tier architecture

In reality, load balancers can potentially be used between any two services with multiple instances within a system's design.

1.3 Services offered by load balancers

LBs not only enable services to be scalable, available, and highly performant, but they also offer some key services like the following:

- **Health checking**: LBs use the heartbeat protocol (The heartbeat protocol is a way of identifying failures in distributed systems. Using this protocol, every node in a cluster periodically reports its health to a monitoring service.) to monitor the health and, therefore, reliability of end-servers. Another advantage of health checking is the improved user experience.
- **TLS termination** (TLS termination, also called TLS/SSL offloading, is the establishment of secure communication channels between clients and servers through encryption/decryption of data.): LBs reduce the burden on end-servers by handling TLS termination with the client.
- **Predictive analytics**: LBs can predict traffic patterns through analytics performed over traffic passing through them or using statistics of traffic obtained over time.
- **Reduced human intervention**: Because of LB automation, reduced system administration efforts are required in handling failures.
- **Service discovery**: An advantage of LBs is that the clients' requests are forwarded to appropriate hosting servers by inquiring about the service registry (Service registry is a repository of the (micro)services and the instances available against each service.).
- **Security**: LBs may also improve security by mitigating attacks like denial-of-service (DoS) (The DoS is an attack where a client floods the server with traffic to exhaust the server's resources (processing and/or memory) such that it is unable to process a legitimate user's request.) at different layers of the OSI model (The open systems interconnection (OSI) model is a conceptual framework that divides the problem of connecting any two machines into seven different layers.) (layers 3, 4, and 7).

As a whole, load balancers provide flexibility (Add or remove machines transparently on the fly.), reliability (Buggy hosts can be removed through health monitoring which makes the system reliable.), redundancy (Multiple paths leading to the same destination or failed server's load is rerouted to the failover machine.), and efficiency (Divide load evenly on all machines to use them effectively from the point of view of the service provider.) to the overall design of the system.
To prevent load balancers from becoming a single point of failure, organizations deploy them in pairs or clusters. These clusters use heartbeat communication to monitor health. When the primary load balancer fails, a backup automatically takes over to keep the service running. In rare cases where the entire cluster fails, engineers can manually reroute traffic to restore availability.
In the coming lessons, we'll see how load balancers can be used in complex applications and which type of load balancer is appropriate for which use case.

2 Global and Local Load Balancing

2.1 Introduction

From the previous lesson, it may seem like load balancing is performed only within the data center. However, load balancing is required at a global and a local scale. Let's understand the function of each of the two:

- **Global server load balancing (GSLB)**: GSLB involves the distribution of traffic load across multiple geographical regions.
- **Local load balancing**: This refers to load balancing achieved within a data center. This type of load balancing focuses on improving efficiency and better resource utilization of the hosting servers in a data center.

Let's understand each of the two techniques below.

2.2 Global server load balancing

GSLB ensures that globally arriving traffic load is intelligently forwarded to a data center. For example, power or network failure in a data center requires that all the traffic be rerouted to another data center. GSLB takes forwarding decisions based on the users' geographic locations, the number of hosting servers in different locations, the health of data centers, and so on.
In the next lesson, we'll also learn how GSLB offers automatic zonal failover. GSLB service can be installed on-premises or obtained through Load Balancing as a Service (LBaaS) (LBaaS is a cloud-based load balancing technique that follows a pay-per-use model. LBaaS offers the flexibility of automatic scaling up or down depending upon the traffic load.).
The illustration below shows that the GSLB can forward requests to three different data centers. Each local load balancing layer within a data center will maintain a control plane connection with the GSLB providing information about the health of the LBs and the server farm. GSLB uses this information to drive traffic decisions and forward traffic load based on each region's configuration and monitoring information.

Now, we'll discuss how the domain name system (DNS) can perform GSLB.

Load balancing in DNS

We understand that DNS can respond with multiple IP addresses for a DNS query. In the lesson on DNS, we discussed that it's possible to do load balancing through DNS while looking at the output of `nslookup`. DNS uses a simple technique of reordering the list of IP addresses in response to each DNS query. Therefore, different users get a reordered IP address list. It results in users visiting a different server to entertain their requests. In this way, DNS distributes the load of requests on different data centers. This is performing GSLB. In particular, DNS uses round-robin to perform load balancing as shown below:

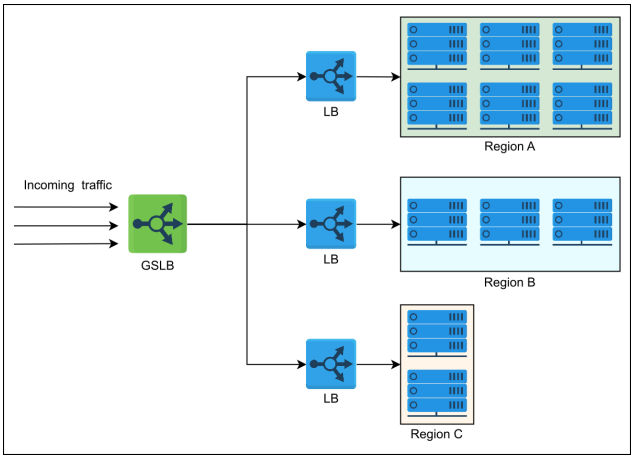
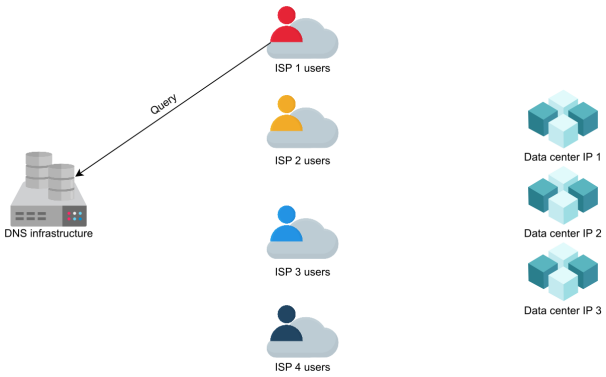
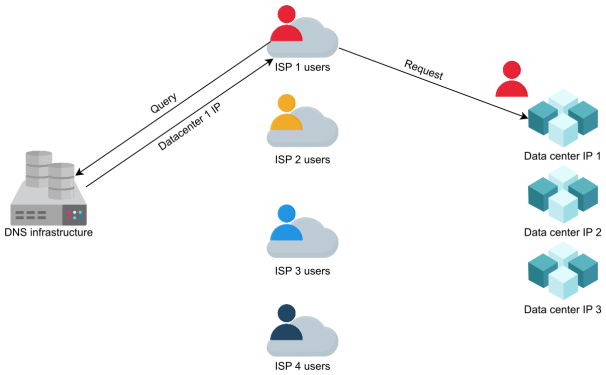


Figure 1.3: Usage of global load balancing to send user requests to different regions

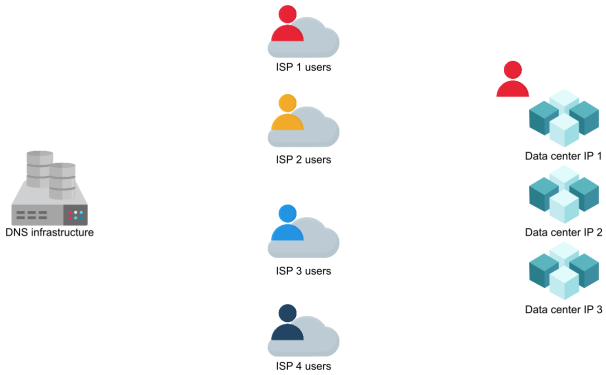


(a) The user from ISP 1 requests the DNS infrastructure for the IP address of a service

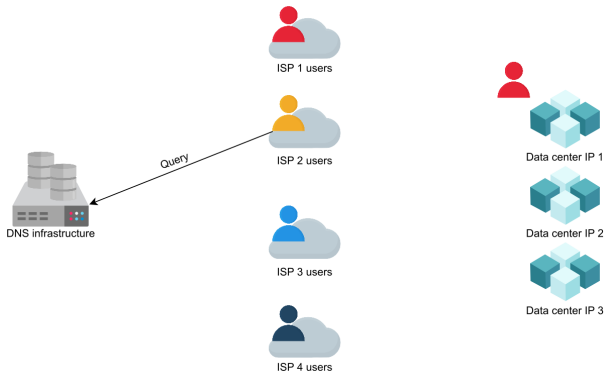
(b) DNS responds with the IP address of data center 1



(a) The user sends a request to data center 1

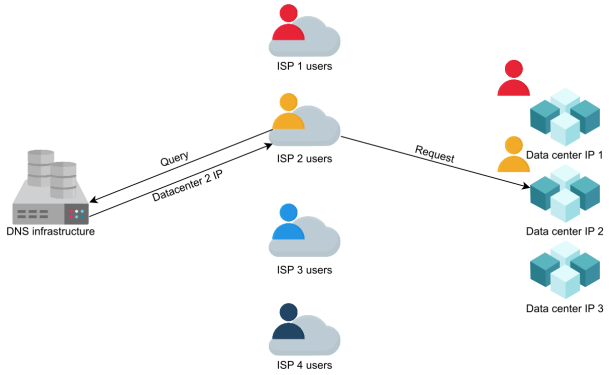


(b) The user from ISP 1 gets served at data center 1



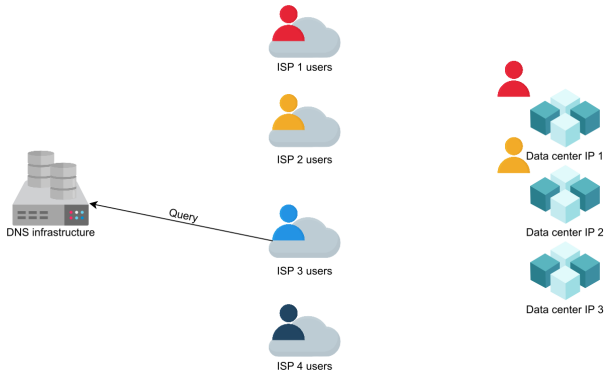
(a) The user from ISP 2 requests the DNS infrastructure for the IP address of the same service

(b) DNS responds with the IP address of data center 2 and does GSLB

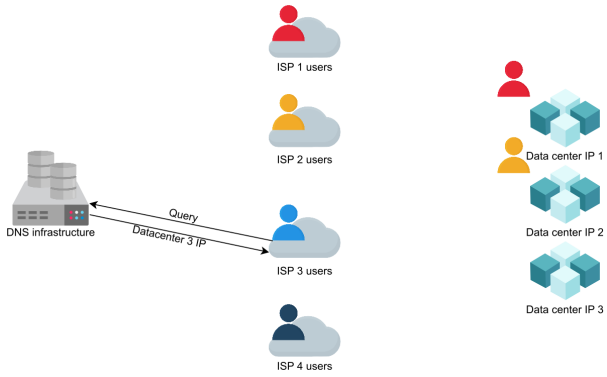


(a) The user sends a request to data center 2

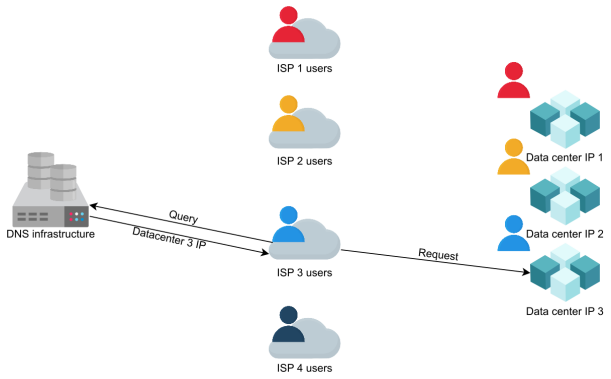
(b) The user from ISP 2 gets served at data center 2



(a) The user from ISP 3 requests the DNS infrastructure for the IP address of the same service

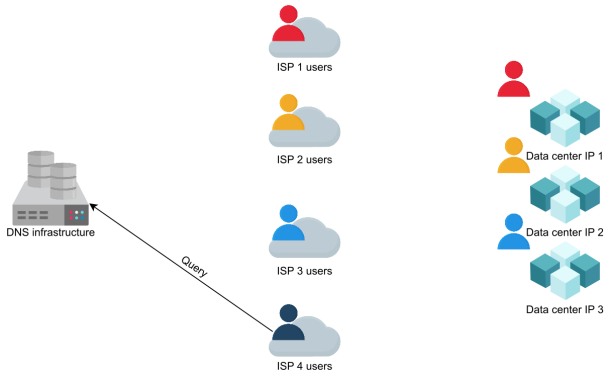


(b) DNS responds with the IP address of data center 3 and does GSLB

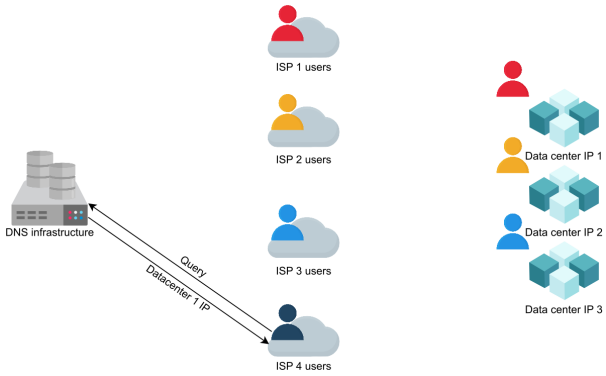


(a) The user sends a request to data center 3

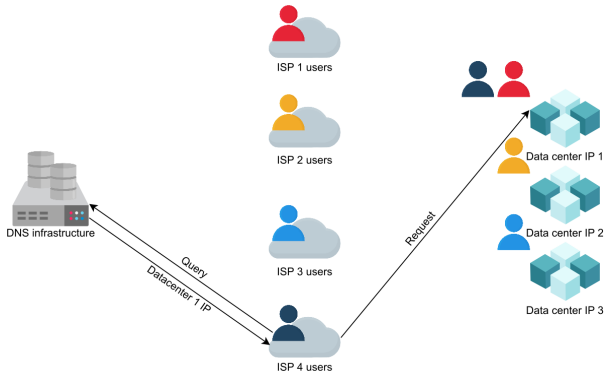
(b) The user from ISP 3 gets served at data center 3



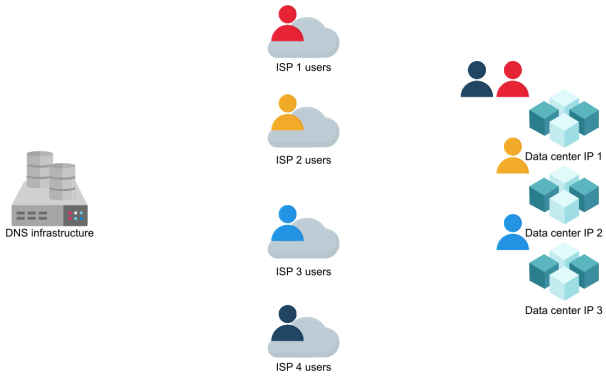
(a) The user from ISP 4 requests the DNS infrastructure for the IP address of the same service



(b) DNS responds with the IP address of data center 1 and does GSLB in round-robin fashion



(a) The user sends a request to data center 1



(b) The user from ISP 4 gets served at data center 1

As shown above, round-robin in DNS forwards clients to data centers in a strict circular order. However, round-robin has the following limitations:

- Different ISPs have a different number of users. An ISP serving many customers will provide the same cached IP to its clients, resulting in uneven load distribution on end-servers.
- Because the round-robin load-balancing algorithm doesn't consider any end-server crashes, it keeps on distributing the IP address of the crashed servers until the TTL of the cached entries expires. Availability of the service, in that case, can take a hit due to DNS-level load balancing.

Despite its limitations, round-robin is still widely used by many DNS service providers. Furthermore , DNS uses short TTL for cached entries to do effective load balancing among different data centers.

> **Note:** DNS isn't the only form of GSLB. Application delivery controllers (ADCs) and cloud-based load balancing (discussed later) are better ways to do GSLB.

Application delivery controllers (ADCs) are part of the application delivery network (ADN). They can be considered the superset of LBs offering various services, including load balancing. The primary task of ADCs is to perform web acceleration to reduce the load from the server farm. Some well-known services between layers 3 and 7 include caching, SSL offloading, proxy/reverse proxy services, IP traffic optimization, and many more. ADCs also implement GSLB.
Anycast is a networking technique where the same IP address is assigned to multiple servers in different locations. Traffic is then routed to the nearest server based on the network's routing table, improving response times. With this in mind, explain the role of anycast in global load balancing and its advantages compared to traditional Global Server Load Balancing (GSLB) mechanisms.
Use the AI assessment widget below to submit your solution and get an interactive response.
Question:
Explain anycast in global load balancing.
Reference Answer:

- **Role of Anycast:**
  - Anycast assigns the same IP address to multiple servers in different locations.
  - Traffic is routed to the nearest server based on network topology, improving response times.
- **Advantages:**
  - Faster failover, as traffic is automatically rerouted if a server becomes unreachable.
  - Improved performance by reducing latency for end users.
  - Simplified DNS configuration since no need for round-robin or geo-aware DNS.
- Anycast is particularly effective for latency-sensitive applications like video streaming, gaming, or real-time communication.

The need for local load balancers

DNS plays a vital role in balancing the load, but it suffers from the following limitations:

- The small size of the DNS packet (512 Bytes) isn't enough to include all possible IP addresses of the servers.

- There's limited control over the client's behavior. Clients may select arbitrarily from the received set of IP addresses. Some of the received IP addresses may belong to busy data centers.
- Clients can't determine the closest address to establish a connection with. Although DNS geolocation and anycasting-like solutions can be implemented, they aren't trivial solutions.
- In case of failures, recovery can be slow through DNS because of the caching mechanism, especially when TTL values are longer.

To solve some of the above problems, we need another layer of load balancing in the form of local LB. In the next lesson, we'll discuss different details about local load balancers.

2.3 What is local load balancing?

Local load balancers reside within a data center. They behave like a reverse proxy (Reverse proxy is a component at the edge of the server-side network that sits between the server and the outside world. It can provide features like privacy, security, caching, etc.) and make their best effort to divide incoming requests among the pool of available servers. Incoming clients' requests seamlessly connect to the LB that uses a virtual IP address (VIP).

> **Quiz:**
> **Question 1:** Can DNS be considered a global server load balancer (GSLB)?
> **Answer:** Yes, DNS can be used as part of global server load balancing (GSLB), but it is typically implemented through custom DNS infrastructure. There are two common approaches to global traffic management (GTM): GTM through
>
> ADCs: Some Application Delivery Controllers (ADCs) support GSLB by forwarding requests based on real-time data such as server health and data center capacity. GTM through DNS: Organizations or DNS providers configure authoritative DNS servers that respond to client requests based on the IP address of the DNS resolver (which is usually close to the user). This allows them to direct traffic to the data center that is geographically or topologically nearest, or based on routing policies such as latency or availability. This is typically achieved via third-party managed DNS providers (e.g., AWS Route 53, Cloudflare) or custom DNS infrastructure that incorporates geolocation and health data.

In the next lesson, we'll explore some advanced details of local load balancers.

3 Advanced Details of Load Balancers

This lesson will focus on some of the well-known algorithms used in the local load balancers. We 'll also understand how load balancers are connected to form a hierarchy, sharing work across different tiers of LBs.

3.1 Algorithms of load balancers

Load balancers distribute client requests according to an algorithm. Some well-known algorithms are given below:

- **Round-robin scheduling**: In this algorithm, each request is forwarded to a server in the pool in a repeating sequential manner.
- **Weighted round-robin**: If some servers have a higher capability of serving clients' requests, then it's preferred to use a weighted round-robin algorithm. In a weighted round-robin algorithm, each node is assigned a weight. LBs forward clients' requests according to the weight of the node. The higher the weight, the higher the number of assignments.

- **Least connections**: In certain cases, even if all the servers have the same capacity to serve clients, uneven load on certain servers is still a possibility. For example, some clients may have a request that requires longer to serve. Or some clients may have subsequent requests on the same connection. In that case, we can use algorithms like least connections, where newer arriving requests are assigned to servers with fewer existing connections. LBs keep a state of the number and mapping of existing connections in such a scenario. We'll discuss more about state maintenance later in the lesson.
- **Least response time**: In performance-sensitive services, algorithms such as least response time are required. This algorithm ensures that the server with the least response time is requested to serve the clients.
- **IP hash**: Some applications provide a different level of service to users based on their IP addresses. In that case, hashing the IP address is performed to assign users' requests to servers.
- **URL hash**: It may be possible that some services within the application are provided by specific servers only. In that case, a client requesting service from a URL is assigned to a certain cluster or set of servers. The URL hashing algorithm is used in those scenarios.

There are other algorithms, such as randomized or weighted least connections algorithms.

Static versus dynamic algorithms

Algorithms can be static or dynamic depending on the machine's state. Let's look at each of the categories individually:

- **Static algorithms** don't consider the changing state of the servers. Therefore, task assignment is carried out based on existing knowledge about the server's configuration. Naturally, these algorithms aren't complex, and they get implemented in a single router or commodity machine where all the requests arrive.
- **Dynamic algorithms** are algorithms that consider the current or recent state of the servers. Dynamic algorithms maintain state by communicating with the server, which adds communication overhead. State maintenance makes the design of the algorithm much more complicated.

Dynamic algorithms require different load balancing servers to communicate with each other to exchange information. Therefore, dynamic algorithms can be modular because no single entity will do the decision-making. Although this adds complexity to dynamic algorithms, it results in improved forwarding decisions. Finally , dynamic algorithms monitor the health of the servers and forward requests to active servers only.

> **Note:** In practice, dynamic algorithms provide far better results because they maintain a state of serving hosts and are, therefore, worth the effort and complexity.

Stateful versus stateless LBs

While static and dynamic algorithms are required to consider the health of the hosting servers, a state is maintained to hold session information of different clients with hosting servers.
If the session information isn't kept at a lower layer (distributed cache or database), load balancers are used to keep the session information. Below , we describe two ways of handling session maintenance through LBs:

- Stateful
- Stateless

Stateful load balancing As the name indicates, **stateful load balancing** involves maintaining a state of the sessions established between clients and hosting servers. The stateful LB incorporates state information in its algorithm to perform load balancing.
Essentially, the stateful LBs retain a data structure that maps incoming clients to hosting servers. Stateful LBs increase complexity and limit scalability because session information of all the clients is maintained across all the load balancers. That is, load balancers share their state information with each other to make forwarding decisions.

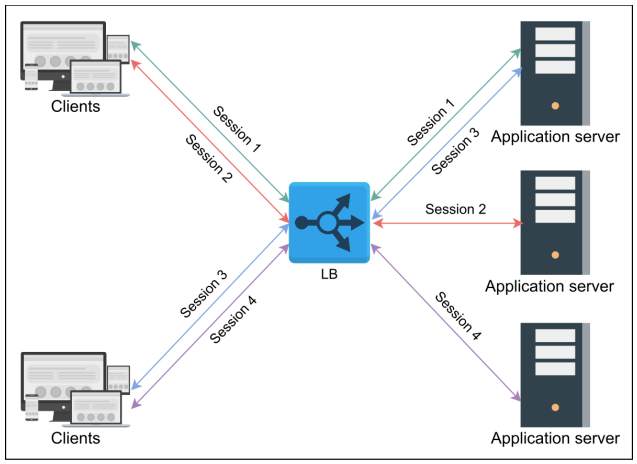


Figure 1.12: Stateful load balancing

Stateless load balancing **Stateless load balancing** maintains no state and is, therefore, faster and lightweight. Stateless LBs use consistent hashing to make forwarding decisions. However, if infrastructure changes (for example, a new application server joining), stateless LBs may not be as resilient as stateful LBs because consistent hashing alone isn't enough to route a request to the correct application server. Therefore, a local state may still be required along with consistent hashing.

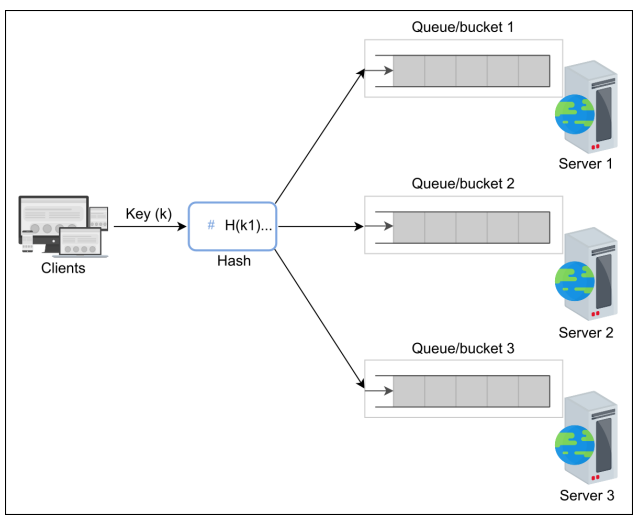


Figure 1.13: Stateless load balancers using hash buckets to map requests to end servers

Therefore, a state maintained across different load balancers is considered as stateful load balancing. Whereas , a state maintained within a load balancer for internal use is assumed as stateless load balancing. Having familiarized yourself with load balancers, consider a scenario where you deploy a website that serves static content, such as text and images. Would you choose a stateful or stateless load balancer, and why?

Question:

Choosing between stateful and stateless load balancer

Reference Answer:

Stateless is likely a good fit – no need to track sessions. Advantages: simplicity, performance, and the potential for easier scalability.

3.2 Types of load balancers

Depending on the requirements, load balancing can be performed at the network/transport and application layer of the open systems interconnection (OSI) (The OSI model is a conceptual framework that classifies different functionalities of a communication system into seven distinct layers. Each layer has its specific set of responsibilities, scope, and interaction with other layers to ensure reliable and efficient communication. For a concise explanation of the OSI model, see https://www.educative.io/answers/what-is-the-osi-model. To dive deep into OSI, see https://www.educative.io/blog/osi-model-layers.) layers.

- **Layer 4 load balancers**: Layer 4 refers to load balancing performed using transport protocols like TCP and UDP. These types of LBs maintain connections/sessions with clients and ensure that the same (TCP/UDP) communication is forwarded to the same back-end server. Even though TLS termination is performed at layer 7 LBs, some layer 4 LBs also support it.
- **Layer 7 load balancers**: Layer 7 load balancers are based on the data of application layer protocols. It's possible to make application-aware forwarding decisions based on HTTP headers, URLs, cookies, and other application-specific data—for example, user ID. Apart from performing TLS termination, these LBs can take responsibilities like rate limiting users, HTTP routing, and header rewriting.

  Note: Layer 7 load balancers are smart in terms of inspection. However, layer 4 load balancers are faster (For instance, Google has developed a high-performance distributed load balancer called Maglev for their cloud platform.) in terms of processing.

3.3 Load balancer deployment

We discussed the trade-offs of load balancing performed at different OSI layers. In practice, however, a single layer LB isn't enough for a large data center. In fact, multiple layers of load balancers coordinate to take informed forwarding decisions. A traditional data center may have a three-tier LB as shown below:

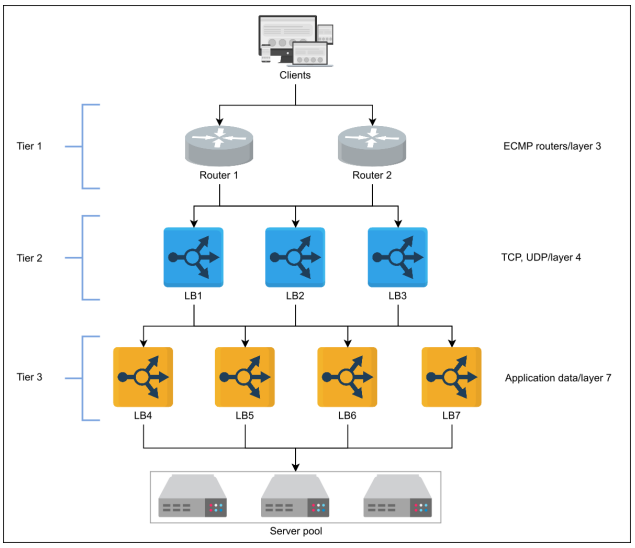


Figure 1.14: Three-tier load balancer in a typical data center

Tier-0 and tier-1 LBs

If DNS can be considered as the tier-0 load balancer, equal cost multipath (ECMP) (Equal-cost multi-path routing (ECMP) is a routing strategy where packet forwarding to a single destination can occur over multiple best paths with equal routing priority. [Wikipedia]) routers are the tier-1 LBs. From the name of ECMP, it's evident that this layer divides incoming traffic on the basis of IP or some other algorithm like round-robin or weighted round-robin. Tier -1 LBs will balance the load across different paths to higher tiers of load

balancers.
ECMP routers play a vital role in the horizontal scalability of the higher-tier LBs.

Tier-2 LBs

The second tier of LBs include layer 4 load balancers. Tier -2 LBs make sure that for any connection, all incoming packets are forwarded to the same tier-3 LBs. To achieve this goal, a technique like consistent hashing can be utilized. But in case of any changes to the infrastructure, consistent hashing may not be enough. Therefore, we have to maintain a local or global state as we'll see in the coming sections of the lesson.
Tier-2 load balancers can be considered the glue between tier-1 and tier-3 LBs. Excluding tier-2 LBs could result in erroneous forwarding decisions in case of failures or dynamic scaling of LBs.

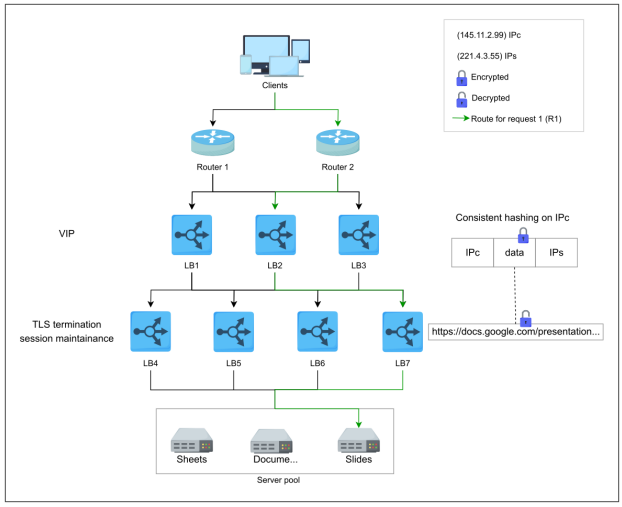
Tier-3 LBs

Layer 7 LBs provide services at tier 3. Since these LBs are in direct contact with the back-end servers, they perform health monitoring of servers at HTTP level. This tier enables scalability by evenly distributing requests among the set of healthy back-end servers and provides high availability by monitoring the health of servers directly. This tier also reduces the burden on end-servers by handling low-level details like TCP-congestion control protocols (TCP makes use of a congestion window to estimate the number of in-flight packets in a connection to avoid network congestion. Two well-known TCP congestion protocols are Tahoe and Reno.), the discovery of Path MTU (maximum transmission unit) (Path MTU is the largest packet size that a source can send on the network beyond which packet fragmentation will happen. This number is equal to the smallest MTU on the network path.), the difference in application protocol between client and back-end servers, and so on. The idea is to leave the computation and data serving to the application servers and effectively utilize load balancing commodity machines for trivial tasks. In some cases, layer 7 LBs are at the same level as the service hosts.

> To summarize, tier 1 balances the load among the load balancers themselves. Tier 2 enables a smooth transition from tier 1 to tier 3 in case of failures, whereas tier 3 does the actual load balancing between back-end servers. Each tier performs other tasks to reduce the burden on end-servers.

Practical example

Let's look at an example where requests from a client come in and get forwarded to different application servers based on the application data inside the client's network packets.



Let's look at the illustration above in the following steps:

1. R_1 indicates request 1 coming through one of the ECMP routers (tier-1 LBs).

2. ECMP routers forward R_1 to any of the three available tier-2 LBs using a round-robin algorithm. Tier -2 LBs take a hash of the source IP address (IP_s) and forward the packet to the next tier of LBs.

3. Tier-3, upon receiving the packet, offloads TLS and reads the HTTP(S) data. By observing the requested URL, it forwards the request to the server handling requests for `slides`.

R_2 takes the same path but a different end-server because the requested URL contains `document` instead of `slides`. Tier-3 LBs are preconfigured to forward requests to application servers based on the application data. For instance, a typical HAProxyHAProxy is an open-source software LB that can perform load balancing based on TCP and HTTP. server can have the following configuration in tier-3 LBs:

HAProxy sample configuration for layer 7 load balancers

```
mode HTTP //define which mode the LB will work on, TCP for tier-2 LBs
acl slidesApp path_end -i /presentation //define a category of applications if
    the path ends in /presentation
use_backend slidesServers if slidesApp // use a set of backend servers if the
    request arrives for slidesApp
backend slidesServers // listing servers serving slidesApp
server slides1 192.168.12.1:80 //using slides1 server to serve slidesApp.
```

Quiz:
Question 1: After a request reaches a back-end server, should the response be routed back through each tier of the load balancers?
Answer: No, the server can send the response directly to the routers (tier-1

LBs) through tier-3 LBs, which can forward the response from the data center. Such a response is called direct routing (DR) or direct server return (DSR). DR/DSR is typically used for non-HTTPS traffic because when TLS termination is involved, responses need to be encrypted before reaching the client, requiring them to pass back through the Layer 7 load balancer for encryption.
Question 2: Why don't the servers directly send the response to the routers (tier-1 LBs) instead of tier-3 LBs?
Answer: Tier-3 LBs maintain some state of connection—for example, SSL

encryption/decryption. This is necessary to give clients a seamless experience.
Question 3: According to you, which tier of LBs is more prone to bugs?
Answer: Tier 3 has more complexity, which makes it more prone to bugs.
Question 4: The image above shows a higher number of tier-3 load balancers than that of tier-2. What do you think is the reason for such a representation?
Answer: Tier 3 performs application-specific analysis and substantially more sophisticated computations. Therefore, handling the same number of queries as tier 2 requires a higher number of machines. Furthermore,

tier-3 LBs maintain the state of a large number of application servers, which may not be possible using the same number of LBs as tier 2.

3.4 Implementation of load balancers

Different kinds of load balancers can be implemented depending on the number of incoming requests, organization, and application-specific requirements:

Hardware load balancers

Load balancers were introduced in the 1990s as hardware devices. Hardware load balancers work as standalone devices and are quite expensive. Nonetheless , they have their performance benefits and are able to

handle a lot of concurrent users. Configuration of hardware-based solutions is problematic because it requires additional human resources. Therefore, they aren't the go-to solutions even for large enterprises that can afford them. Availability can be an issue with hardware load balancers because additional hardware will be required to failover to in case of failures. Finally , hardware LBs can have higher maintenance/operational costs and compatibility issues making them less flexible. Not to mention that hardware LBs have vendor locks as well.

Software load balancers

Software load balancers are becoming increasingly popular because of their flexibility, programmability, and cost-effectiveness. That 's all possible because they're implemented on commodity hardware. Software LBs scale well as requirements grow. Availability won't be an issue with software LBs because small additional costs are required to implement shadow load balancers on commodity hardware. Additionally , software LBs can provide predictive analysis that can help prepare for future traffic patterns.

Cloud load balancers

With the advent of the field of cloud computing, Load Balancers as a Service (LBaaS) has been introduced. This is where cloud owners provide load balancing services. Users pay according to their usage or the service-level agreement (SLA) with the cloud provider. Cloud -based LBs may not necessarily replace a local on-premise load balancing facility, but they can perform global traffic management between different zones. Primary advantages of such load balancers include ease of use, management, metered cost, flexibility in terms of usage, auditing, and monitoring services to improve business decisions. An example of how cloud-based LBs can provide GSLB is given below:

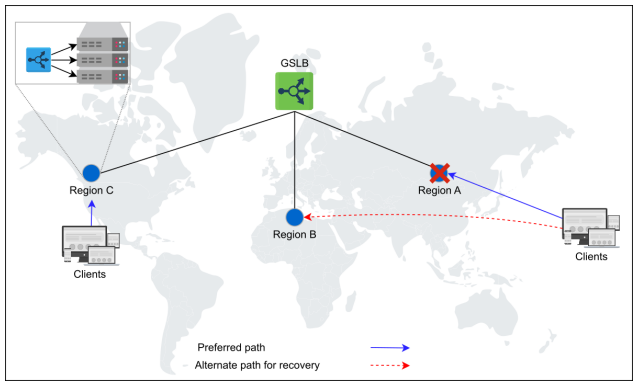


Figure 1.15: GSLB is obtained through LBaaS, and the regions contain data centers that are the property of application providers

> **Note:** Another interesting implementation of load balancers comes in the form of **client-side load balancing**. Client-side load balancing is suited where there are numerous services, each with many instances (such as load balancing in Twitter). Our focus, however, is on traditional load balancers because most three-tier applications employ these in their design.

3.5 Conclusion

LBs have come a long way to become a service offered in the cloud, starting since their inception in the form of hardware. They 're a key component of any enterprise-level service. Horizontal scalability of hosting servers will always require a good load balancing layer capable of providing load balancing, session maintenance, TLS offloading, service discovery, and more.
In this chapter, we have explored various functions of load balancing. In addition to different roles, load balancers are often responsible for mitigating distributed denial-of-service (DDoS) attacks. How can they distinguish between legitimate traffic and malicious traffic during such incidents?

Use the AI assessment widget below to submit your solution and get an interactive response.
Question:
How can load balancers differentiate requests?
Reference Answer:
Load balancers differentiate between legitimate and malicious traffic during DDoS attacks by:

- Analyzing traffic patterns for anomalies, such as an unusually high number of requests from a single IP or range.
- Using **rate-limiting** to cap the number of requests allowed from a specific source.
- Implementing **challenge-response protocols** (e.g., CAPTCHAs) to verify human users.
- Leveraging threat intelligence to block known malicious IPs or regions.
- Using **behavioral analytics** and machine learning models to identify and filter out traffic resembling attack patterns. These techniques will be used in tandem to provide comprehensive protection against DDoS attacks. However, IP-based solutions can be circumvented by attackers using IP spoofing, proxy servers, or botnets that distribute attacks across many different IP addresses, making it harder to identify and block malicious traffic based on IP alone.